ΗΥ240: Δομές Δεδομένων
Χειμερινό Εξάμηνο - Ακαδημαϊκό Έτος 2024- 25
Διδάσκουσα: Παναγιώτα Φατούρου

Προγραμματιστική Εργασία - 2ο Μέρος

Ημερομηνία Παράδοσης: Παρασκευή, 20 Δεκεμβρίου 2024, ώρα 23:59.

Τρόπος Παράδοσης: Μέσω της σελίδες elearn του μαθήματος. Πληροφορίες σχετικά με το elearn παρέχονται στην ιστοσελίδα του μαθήματος.

Γενική Περιγραφή

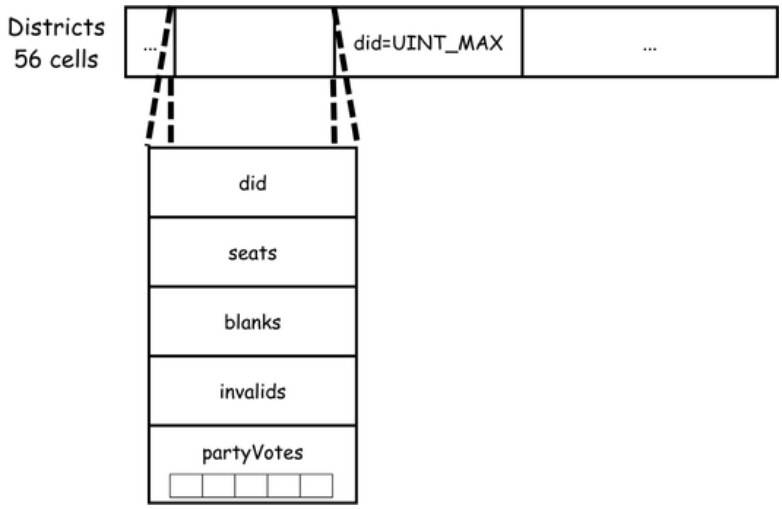
Στην εργασία αυτή καλείστε να υλοποιήσετε ένα πρόγραμμα που προσομοιώνει το εκλογικό σύστημα της Ελλάδας. Το **ελληνικό εκλογικό σύστημα** είναι το σύστημα με το οποίο κατανέμονται οι βουλευτικές έδρες στα κόμματα με βάση τις ψήφους που έλαβαν στις βουλευτικές εκλογές. Ρυθμίζεται από ειδικό νόμο, που ονομάζεται εκλογικός νόμος.

Σημείωση: Προκειμένου να πραγματωθούν οι εκπαιδευτικοί στόχοι της εργασίας, δεν ακολουθείται πιστά ο εκλογικός νόμος και έχουν γίνει σημαντικές απλοποιήσεις σχετικά με τον τρόπο με τον οποίο κατανέμονται οι έδρες. Συνεπώς, όσα αναγράφονται στην παρούσα εκφώνηση δεν αντιστοιχούν με ακρίβεια στην ισχύουσα εκλογική νομοθεσία.

Αναλυτική Περιγραφή Ζητούμενης Υλοποίησης

Για την υλοποίηση της προσομοίωσης του ελληνικού εκλογικού συστήματος, θα πρέπει να χρησιμοποιήσετε έναν πίνακα σταθερής χωρητικότητας **56 θέσεων**, ο οποίος ονομάζεται **πίνακας εκλογικών περιφερειών (Districts)** (Σχήμα 1). Ο πίνακας εκλογικών περιφερειών είναι struct με μοναδικό πεδίο **data**, έναν πίνακα από εγγραφές (structs) τύπου District. Κάθε στοιχείο του πίνακα εκλογικών περιφερειών (Districts) είναι μια εγγραφή τύπου **District** με τα ακόλουθα πεδία:

- **did:** Αναγνωριστικό (int) που χαρακτηρίζει μοναδικά την περιφέρεια.
- **seats:** Αριθμός (int) των βουλευτικών εδρών που αντιστοιχούν στην περιφέρεια.
- **blanks:** Αριθμός (int) των **λευκών** ψήφων που μετρήθηκαν κατά την καταμέτρηση ψήφων σε όλα τα εκλογικά τμήματα της περιφέρειας.
- **invalids:** Αριθμός (int) των **άκυρων** ψήφων που μετρήθηκαν κατά την καταμέτρηση ψήφων σε όλα τα εκλογικά τμήματα της περιφέρειας.
- **partyVotes:** Πίνακας (int[]) μεγέθους ίσο με τον αριθμό των κομμάτων που συμμετέχουν στις εκλογές (σ' αυτήν την εργασία θα θεωρήσετε πως το μέγεθος αυτό είναι 5). Τα αναγνωριστικά κομμάτων (pid) χρησιμοποιούνται για τη διευθυνσιοδότηση του πίνακα. Στο στοιχείο partyVotes[j], όπου $0 \leq j \leq 4$, αποθηκεύεται ο αριθμός των ψήφων που έχει λάβει το κόμμα με αναγνωριστικό j στην εκλογική περιφέρεια με αναγνωριστικό did.



Σχήμα 1 Πίνακας εκλογικών περιφερειών (Districts).

Για την αποθήκευση των εκλογικών τμημάτων όλων των περιφερειών θα πρέπει να υλοποιήσετε έναν πίνακα κατακερματισμού, ο οποίος ονομάζεται **πίνακας κατακερματισμού εκλογικών τμημάτων (Stations)** (Σχήμα 2). Επομένως, στη δεύτερη φάση της προγραμματιστικής εργασίας, δεν υπάρχει πλέον μια λίστα εκλογικών τμημάτων ανά περιφέρεια, αλλά μια συγκεντρωτική δομή, που είναι ένας πίνακας κατακερματισμού και αποθηκεύει πληροφορίες για όλα τα εκλογικά τμήματα ανεξάρτητα από την εκλογική περιφέρεια (District) στην οποία αυτά ανήκουν.

Για την επίλυση των συγκρούσεων θα πρέπει να ακολουθήσετε τη μέθοδο των ταξινομημένων αλυσίδων. Επομένως, σε κάθε κελί του πίνακα κατακερματισμού εκλογικών τμημάτων (Stations) αποθηκεύεται μια αλυσίδα, κάθε στοιχείο της οποίας αντιστοιχεί σε ένα **εκλογικό τμήμα (Station)**.

Ο **πίνακας κατακερματισμού εκλογικών τμημάτων (Stations)** είναι struct με πεδία:

- **capacity:** Αριθμός (int) που αναπαριστά το μέγεθος της χωρητικότητας του πίνακα κατακερματισμού.
- **data:** Δείκτης (StationsList*) σε δυναμικό πίνακα μεγέθους capacity, ταξινομημένων αλυσίδων εκλογικών τμημάτων. Κάθε αλυσίδα εκλογικών τμημάτων είναι struct με μοναδικό πεδίο το **begin**, έναν δείκτη (Station*) στο πρώτο στοιχείο της αλυσίδας. Κάθε κόμβος αλυσίδας εκλογικών τμημάτων είναι μια εγγραφή (struct) τύπου Station.

Κάθε εγγραφή τύπου **Station** αποτελείται από τα παρακάτω πεδία:

- **sid:** Αναγνωριστικό (int) που χαρακτηρίζει μοναδικά το εκλογικό τμήμα.
- **did:** Αναγνωριστικό (int) της εκλογικής περιφέρειας (District) στην οποία ανήκει το εκλογικό τμήμα.
- **registered:** Αριθμός (int) των ψηφοφόρων που είναι εγγεγραμμένοι στο εκλογικό τμήμα.
- **voters:** Δένδρο ψηφοφόρων (VotersTree) εκλογικού τμήματος με αναγνωριστικό sid (το οποίο περιγράφεται πιο αναλυτικά στη συνέχεια). Είναι struct με μοναδικό πεδίο το **root**, έναν δείκτη (Voter*) στη ρίζα του δέντρου. Κάθε κόμβος αυτού του δένδρου είναι μια εγγραφή (struct) τύπου Voter.
- **next:** Δείκτης (Station*) στον επόμενο κόμβο της αλυσίδας που ανήκει το εκλογικό τμήμα με αναγνωριστικό sid.

Τη χωρητικότητα του πίνακα κατακερματισμού εκλογικών τμημάτων το επιλέγετε εσείς και θα πρέπει να είστε σε θέση να δικαιολογήσετε την επιλογή σας. Για την υλοποίηση της συνάρτησης κατακερματισμού θα πρέπει να χρησιμοποιήσετε τη μέθοδο του καθολικού κατακερματισμού.

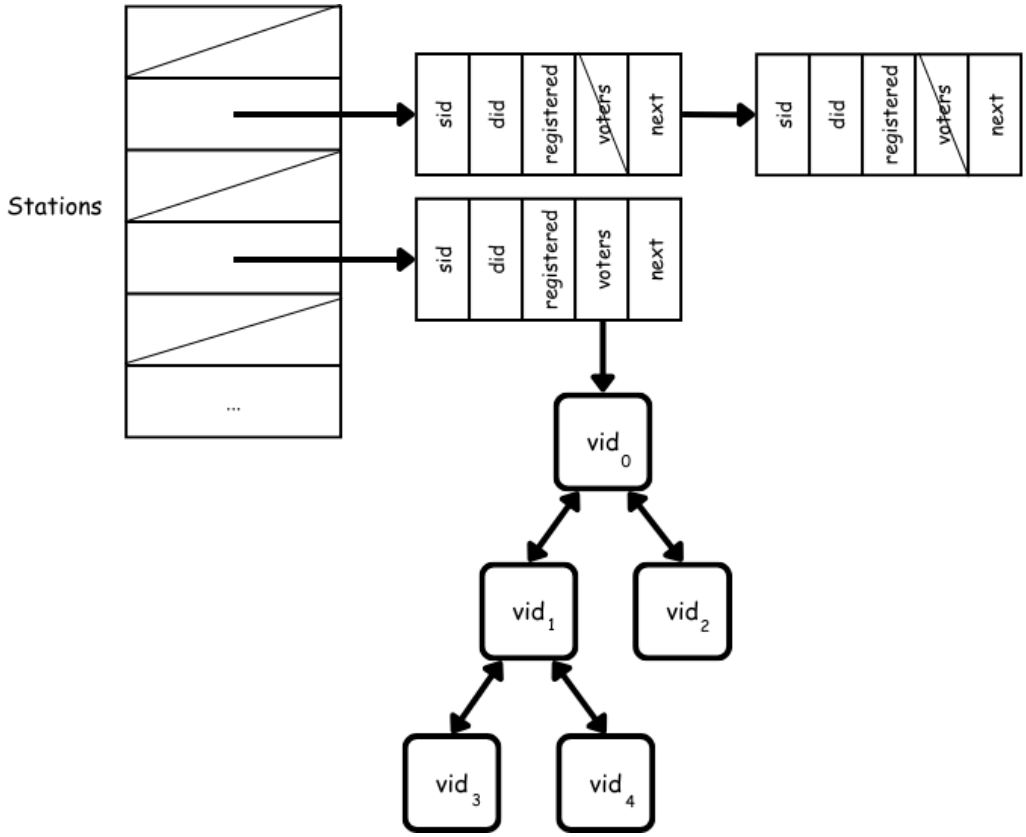
Για την επιλογή της χωρητικότητάς του πίνακα και την υλοποίηση του καθολικού κατακερματισμού δίνονται τα εξής:

- **Primes:** Πίνακας (int[]) πρώτων αριθμών σε αύξουσα σειρά.
- **ParsedMaxStationsCount:** Αριθμός (int) που αναπαριστά το μέγιστο πλήθος των εκλογικών τμημάτων.
- **ParsedMaxSid:** Αριθμός (int) που αναπαριστά το μέγιστο αναγνωριστικό εκλογικού τμήματος (sid).

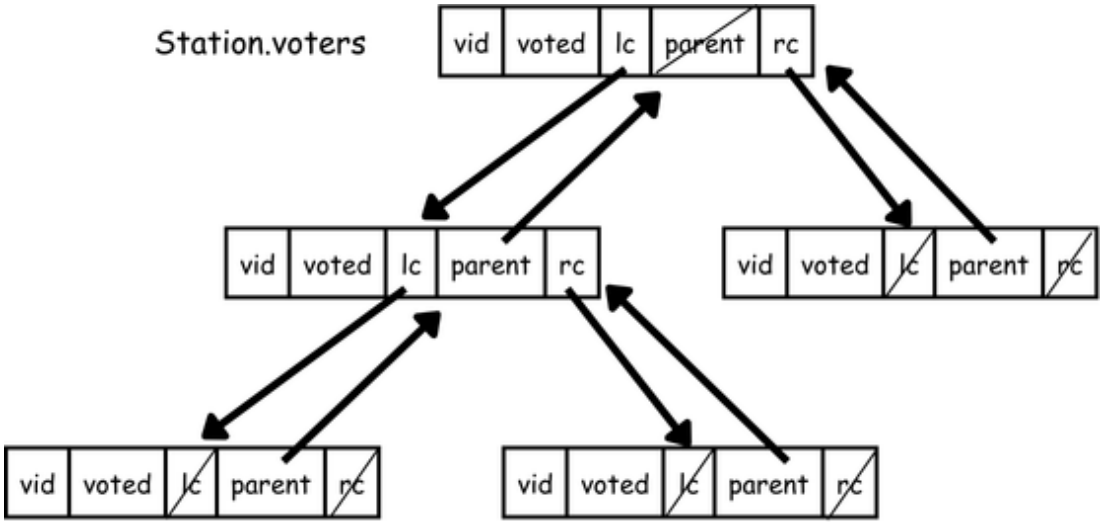
Αυτές οι μεταβλητές είναι **global**, έχουν δηλωθεί στα voting.h, voting.c και αρχικοποιούνται στη main, βάσει τιμών που αναγράφονται στις πρώτες γραμμές κάθε testfile.

Το **δένδρο ψηφοφόρων (voters)** του εκλογικού τμήματος με αναγνωριστικό sid (που αναφέρθηκε παραπάνω) είναι **ένα πλήρες, μη-ταξινομημένο, διπλά-συνδεδεμένο δυαδικό δένδρο** (Σχήμα 3). Κάθε στοιχείο αυτού του δένδρου αντιστοιχεί σε έναν ψηφοφόρο που είναι εγγεγραμμένος στο εκλογικό τμήμα με αναγνωριστικό sid και αποτελεί μια εγγραφή τύπου **Voter** με τα ακόλουθα πεδία:

- **vid:** Αναγνωριστικό (int) που χαρακτηρίζει μοναδικά τον ψηφοφόρο.
- **voted:** Μεταβλητή (bool) που δηλώνει αν ο ψηφοφόρος έχει εξασκήσει το δικαίωμα ψήφου.
- **parent:** Δείκτης (Voter*) στον πατρικό κόμβο.
- **lc:** Δείκτης (Voter*) στο αριστερό παιδί του κόμβου.
- **rc:** Δείκτης (Voter*) στο δεξί παιδί του κόμβου.



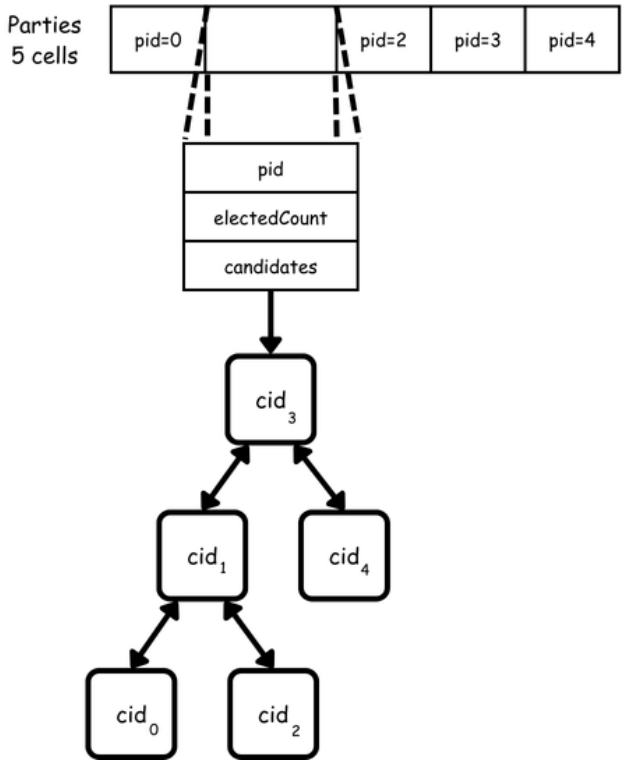
Σχήμα 2 Πίνακας κατακερματισμού εκλογικών τμημάτων (Stations).



Σχήμα 3 Δένδρο ψηφοφόρων (Station.voters) πλήρες, μη-ταξινομημένο, διπλά-συνδεδεμένο δυαδικό δένδρο.

Για τις ανάγκες της εργασίας αυτής, θεωρούμε ότι υπάρχουν πέντε διαφορετικά κόμματα που συμμετέχουν στις εκλογές. Θα πρέπει να χρησιμοποιήσετε έναν πίνακα σταθερής χωρητικότητας **5 θέσεων**, ο οποίος ονομάζεται **πίνακας κομμάτων (Parties)** (Σχήμα 4). Ο πίνακας κομμάτων (Parties) είναι struct με μοναδικό πεδίο **data**, έναν πίνακα από εγγραφές (structs) τύπου Party. Κάθε στοιχείο του πίνακα κομμάτων είναι μια εγγραφή τύπου **Party** με τα ακόλουθα πεδία:

- **pid:** Αναγνωριστικό (int) που χαρακτηρίζει το κόμμα.
- **electedCount:** Αριθμός (int) των βουλευτών που εκλέγει το κόμμα.
- **candidates:** Δένδρο υποψήφιων (candidates) κόμματος με αναγνωριστικό pid (το οποίο περιγράφεται πιο αναλυτικά στη συνέχεια). Είναι struct με μοναδικό πεδίο το **root**, έναν δείκτη (Candidate*) στη ρίζα του δέντρου. Κάθε κόμβος αυτού του δένδρου είναι μια εγγραφή (struct) τύπου Candidate.

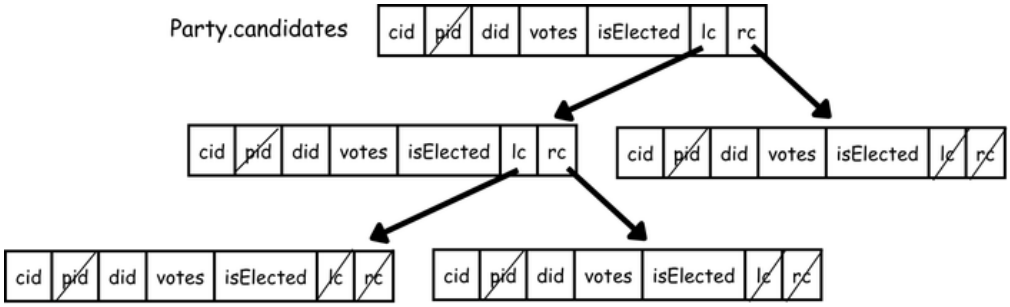


Σχήμα 4 Πίνακας κομμάτων (Parties).

Το **δένδρο υποψήφιων (candidates)** του κόμματος αποθηκεύει πληροφορίες για τους υποψήφιους βουλευτές ενός κόμματος. Είναι ένα **απλά συνδεδεμένο δένδρο δυαδικής αναζήτησης(Binary Search Tree), ταξινομημένο** βάσει του αναγνωριστικού του υποψηφίου (cid) (Σχήμα 5). Κάθε στοιχείο του δένδρο υποψήφιων (candidates) κόμματος pid, αποτελεί μια εγγραφή τύπου **Candidate** και αντιστοιχεί σε έναν υποψήφιο βουλευτή του κόμματος με αναγνωριστικό pid. Κάθε εγγραφή τύπου Candidate αποτελείται από τα παρακάτω πεδία :

- **cid:** Αναγνωριστικό (int) που χαρακτηρίζει τον υποψήφιο βουλευτή.
- **pid:** Αναγνωριστικό (int) που αντιστοιχεί στο κόμμα (Party), το οποίο ο υποψήφιος εκπροσωπεί.
- **did:** Αναγνωριστικό (int) της εκλογικής περιφέρειας (District), στην οποία ο υποψήφιος εκλέγεται βουλευτής.
- **votes:** Αριθμός (int) των ψήφων που συγκέντρωσε ο υποψήφιος από όλα τα εκλογικά τμήματα της εκλογικής περιφέρειας με αναγνωριστικό did στην οποία είναι υποψήφιος.
- **isElected:** Μεταβλητή (bool) που δηλώνει αν ο υποψήφιος έχει εκλεγεί βουλευτής.
- **lc:** Δείκτης (Candidate*) στο αριστερό παιδί του κόμβου.

- **rc:** Δείκτης (Candidate*) στο δεξί παιδί του κόμβου.

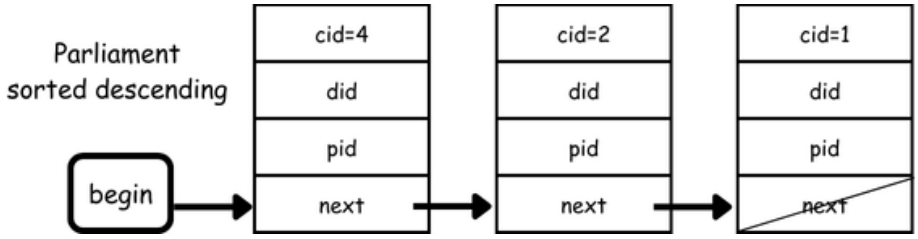


Σχήμα 5 Δένδρο υποψηφίων (Party.candidates) απλά-συνδεδεμένο δυαδικό δένδρο αναζήτησης, ταξινομημένο ως προς το αναγνωριστικό του κάθε υποψηφίου.

Τέλος, θα πρέπει να υλοποιήσετε τη δομή του κοινοβουλίου η οποία περιέχει τους εκλεγμένους βουλευτές. Ονομάζεται **κοινοβούλιο (Parliament)** και είναι μια **ταξινομημένη, απλά συνδεδεμένη λίστα, σε φθίνουσα διάταξη ως προς το αναγνωριστικό του βουλευτή (cid)** (Σχήμα 6). Το κοινοβούλιο είναι struct με μοναδικό πεδίο το **begin**, έναν δείκτη (ElectedCandidate*) στο πρώτο στοιχείο της λίστας. Κάθε κόμβος αυτής της λίστας αποτελεί μια εγγραφή τύπου **ElectedCandidate** με πεδία:

- **cid:** Αναγνωριστικό (int) που χαρακτηρίζει τον υποψήφιο που έχει εκλεγεί βουλευτής.
- **pid:** Αναγνωριστικό (int) που αντιστοιχεί στο κόμμα (Party) το οποίο ο βουλευτής εκπροσωπεί.
- **did:** Αναγνωριστικό (int) της εκλογικής περιφέρειας (District) στην οποία ο βουλευτής έχει εκλεγεί.
- **next:** Δείκτης (ElectedCandidates*) στον επόμενο κόμβο της λίστας

Η δομή του **κοινοβουλίου (Parliament)** δεν περιέχει αρχικά στοιχεία.



Σχήμα 6 Κοινοβούλιο (Parliament) απλά συνδεδεμένη λίστα, ταξινομημένη σε φθίνουσα σειρά.

Οι αρχικές τιμές των παραπάνω δομών περιγράφονται συγκεντρωτικά στην Ενότητα (Αρχικές τιμές πεδίων δομών δεδομένων) στο τέλος της εκφώνησης της εργασίας.

Τρόπος Λειτουργίας Προγράμματος

Το πρόγραμμα που θα δημιουργηθεί θα πρέπει να εκτελείται καλώντας την ακόλουθη εντολή:

<executable> <input_file>

όπου <executable> είναι το όνομα του εκτελέσιμου αρχείου του προγράμματος (π.χ . a.out) και <input_file > είναι το όνομα ενός αρχείου εισόδου (π.χ. testfile) το οποίο περιέχει γεγονότα των ακόλουθων μορφών:

– **A**

Γεγονός τύπου *announce elections* το οποίο υποδηλώνει την αναγγελία των εκλογών. Κατά το γεγονός αυτό αρχικοποιούνται οι δομές:

- πίνακας εκλογικών περιφερειών (Districts)
- πίνακας κατακερματισμού εκλογικών τμημάτων (Stations)
- πίνακας κομμάτων (Parties)
- κοινοβούλιο (Parliament)

Οι αρχικές τιμές των παραπάνω δομών περιγράφονται συγκεντρωτικά στην Ενότητα (Αρχικές τιμές πεδίων δομών δεδομένων) στο τέλος της εκφώνησης της εργασίας.

Μετά το πέρας της εκτέλεσης ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
A
DONE
```

– **D <did> <seats>**

Γεγονός τύπου *create district* το οποίο σηματοδοτεί τη δημιουργία μιας νέας εκλογικής περιφέρειας (District) με αναγνωριστικό <did> και αριθμό εκλεγόμενων εδρών <seats>. Η νέα εκλογική περιφέρεια αρχικοποιείται κατάλληλα και εισάγεται στον πίνακα εκλογικών περιφερειών. **Η χρονική πολυπλοκότητα της εισαγωγής πρέπει να είναι O(log n)** (επομένως θα πρέπει να υλοποιήσετε αλγόριθμο ώστε να βρίσκεται το πρώτο κενό κελί του πίνακα Districts σε O(log n) χρόνο). Μετά το πέρας της εκτέλεσης ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
D <did> <seats>
Districts
<did1>, <did2>, ..., <didn>
DONE
```

όπου n είναι ο αριθμός των εκλογικών περιφερειών που έχουν δημιουργηθεί μέχρι τώρα και για κάθε i ∈ {1, ..., n}, $<did_i>$ είναι το αναγνωριστικό της i-οστής εκλογικής περιφέρειας.

– **S <sid> <did>**

Γεγονός τύπου *create station* το οποίο σηματοδοτεί τη δημιουργία ενός νέου εκλογικού τμήματος (Station) με αναγνωριστικό <sid> στην εκλογική περιφέρεια με αναγνωριστικό

<did>. Η εισαγωγή του εκλογικού τμήματος γίνεται στην κατάλληλη αλυσίδα του πίνακα κατακερματισμού, βάσει της τιμής κατακερματισμού του αναγνωριστικού του <sid>. Μετά το πέρας της εκτέλεσης ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
S <sid> <did>
Stations[h]
<sid1>, <sid2>, ..., <sidn>
DONE
```

όπου h είναι η τιμή κατακερματισμού του <sid>, n είναι ο αριθμός των εκλογικών τμημάτων της συγκεκριμένης αλυσίδας και για κάθε i ∈ {1, …, n} $<sid_i>$ είναι το αναγνωριστικό του i -οστού εκλογικού τμήματος στην αλυσίδα αυτή.

– **C <cid> <pid> <did>**

Γεγονός τύπου *register candidate* το οποίο σηματοδοτεί την εγγραφή ενός νέου υποψηφίου (Candidate) με αναγνωριστικό <cid> του κόμματος με αναγνωριστικό <pid> στην εκλογική περιφέρεια με αναγνωριστικό <did>. Το γεγονός αυτό προσθέτει το νέο υποψήφιο στο δένδρο υποψήφιων του κόμματος με αναγνωριστικό <pid>. Η χρονική πολυπλοκότητα του γεγονότος αυτού πρέπει να είναι O(log n), όπου n είναι το μέγεθος του δένδρου (δηλαδή ο αριθμός των κόμβων του). Μετά το πέρας της εκτέλεσης ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
C <cid> <pid> <did>
Candidates[pid]
<cid1> <did1>,
<cid2> <did2>,
…,
<cidn> <didn>
DONE
```

όπου <pid> είναι το αναγνωριστικό του κόμματος που ανήκει ο υποψήφιος, n είναι το πλήθος των υποψηφίων στο δένδρο υποψήφιων του κόμματος <pid> και για κάθε i ∈ {1, …, n}:

- $<cid_i>$ είναι το αναγνωριστικό του i-οστού υποψηφίου (σύμφωνα με την ενδοδιατεταγμένη διάσχιση) στο δένδρο υποψήφιων του κόμματος <pid>,
- $<did_i>$ είναι το αναγνωριστικό της περιφέρειας στην οποία εκλέγεται ο υποψήφιος με αναγνωριστικό $<cid_i>$

Προσοχή: Για την εκτύπωση των αναγνωριστικών των υποψηφίων πρέπει να εφαρμόζεται ενδοδιατεταγμένη διάσχιση στο δένδρο υποψήφιων του κόμματος <pid>.

– **R <vid> <sid>**

Γεγονός τύπου *register voter* το οποίο σηματοδοτεί την εγγραφή ενός νέου ψηφοφόρου (Voter) με αναγνωριστικό <vid> στο εκλογικό τμήμα με αναγνωριστικό <sid>. Το γεγονός

αυτό προσθέτει το νέο ψηφοφόρο στο δένδρο ψηφοφόρων του εκλογικού τμήματος με αναγνωριστικό <sid>.

Προσοχή: Η εισαγωγή θα πρέπει να γίνεται με τρόπο ώστε το δένδρο να παραμένει πλήρες (με τρόπο συμβατό με την **Άσκηση 3 της 3ης Σειράς Ασκήσεων**). Η χρονική πολυπλοκότητα της λειτουργίας αυτής πρέπει να είναι O(log n), όπου n είναι το πλήθος των στοιχείων στο δένδρο.

Επιπλέον σε αυτό το σημείο αυξάνεται το πλήθος (πεδίο registered) των εγγεγραμμένων ψηφοφόρων του εκλογικού τμήματος <sid> κατά ένα. Μετά το πέρας της εκτέλεσης ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
R <vid> <sid>
Voters[sid]
<vid1>, <vid2>, ..., <vidn>
DONE
```

όπου n είναι το πλήθος των ψηφοφόρων του εκλογικού τμήματος με αναγνωριστικό <sid> και για κάθε i ∈ {1, …, n}, $<vid_i>$ είναι το αναγνωριστικό του i-οστού ψηφοφόρου στο δένδρο ψηφοφόρων του εκλογικού τμήματος με αναγνωριστικό <sid>, στη διάταξη που προκύπτει όταν εφαρμόζεται ενδοδιατεταγμένη διάσχιση στο δένδρο ψηφοφόρων του εκλογικού τμήματος με αναγνωριστικό <sid>.

– **V <vid> <sid> <cid> <pid>**

Γεγονός τύπου *vote* το οποίο σηματοδοτεί τη διαδικασία ψήφου ενός ψηφοφόρου . Ο ψηφοφόρος (Voter) με αναγνωριστικό <vid> που ανήκει στο εκλογικό τμήμα (Station) με αναγνωριστικό <sid> ψηφίζει τον υποψήφιο (Candidate) με αναγνωριστικό <cid> (ή λευκό, ή άκυρο) που ανήκει στο κόμμα (Party) με αναγνωριστικό <pid>.

Το γεγονός αυτό εντοπίζει πρώτα το εκλογικό τμήμα με αναγνωριστικό <sid> στον πίνακα κατακερματισμού εκλογικών τμημάτων. Στη συνέχεια εντοπίζει τον ψηφοφόρο με αναγνωριστικό <vid> στη δένδρο ψηφοφόρων του εκλογικού του τμήματος και σημειώνει ότι συμμετείχε στις εκλογές (θέτοντας σε true το πεδίο voted).

Σε αυτό το σημείο προσμετράτε και η ψήφος ως εξής:

- Αν το <cid> είναι ίσο με το δεσμευμένο αναγνωριστικό -1, τότε αυξάνεται κατά ένα ο αριθμός των λευκών της περιφέρειας (πεδίο blanks) στην οποία ανήκει το εκλογικό τμήμα του ψηφοφόρου.
- Ομοίως, αν το <cid> είναι ίσο με το δεσμευμένο αναγνωριστικό -2, τότε αυξάνεται κατά ένα ο αριθμός των άκυρων της περιφέρειας (πεδίο invalids) στην οποία ανήκει το εκλογικό τμήμα του ψηφοφόρου.
- Σε κάθε άλλη περίπτωση εντοπίζεται ο υποψήφιος με αναγνωριστικό <cid> στο δένδρο υποψηφίων του κόμματος <pid> και αυξάνεται ο αριθμός των ψήφων του (πεδίο votes στο struct Candidate).

Επιπλέον, σε αυτό το σημείο, θα πρέπει να αυξήσουμε τον αριθμό των ψήφων που έχει λάβει το κόμμα στο οποίο ανήκει ο υποψήφιος <cid> στη συγκεκριμένη εκλογική περιφέρεια, δηλαδή θα πρέπει να αυξάνεται κατά ένα ο μετρητής που είναι αποθηκευμένος στο κατάλληλο στοιχείο <pid> του πίνακα partyVotes της εκλογικής περιφέρειας στην οποία ανήκει το εκλογικό τμήμα του υποψήφιου.

Μετά το πέρας της εκτέλεσης ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
V <vid> <sid> <cid> <pid>
District[<did>]
Blanks
<blanks>
invalids
<invalids>
partyVotes:
<pid1> <votes1>,
<pid2> <votes2>,
…,
<pidk> <votesk>
DONE
```

όπου <did> είναι το αναγνωριστικό της εκλογικής περιφέρειας στην οποία εκλέγεται ο υποψήφιος με αναγνωριστικό <cid>, k είναι ο αριθμός των κομμάτων που συμμετέχουν στις εκλογές και για κάθε i ∈ {1, …, k}, <pid_i> είναι το αναγνωριστικό του i-οστού κόμματος και <$votes_i$> το πλήθος των ψήφων που έλαβε το κόμμα με αναγνωριστικό <pid_i> στην εκλογική περιφέρεια, στην οποία ανήκει ο υποψήφιος με αναγνωριστικό <cid>. Επίσης, <blanks> είναι ο αριθμός των λευκών και <invalids> ο αριθμός των άκυρων ψήφων στη συγκεκριμένη εκλογική περιφέρεια.

– **M <did>**

Γεγονός τύπου *count votes* το οποίο σηματοδοτεί την καταμέτρηση ψήφων στην εκλογική περιφέρεια με αναγνωριστικό <did>. Το γεγονός αυτό εκτελεί τις ακόλουθες ενέργειες για την εκλογική περιφέρεια με αναγνωριστικό <did>:

1. Υπολογίζεται το **εκλογικό μέτρο**, το οποίο ορίζεται ως ο λόγος του συνόλου των **έγκυρων ψήφων** στην εκλογική περιφέρεια (δηλαδή το άθροισμα των ψήφων όλων των κομμάτων στην εκλογική περιφέρεια χωρίς τα λευκά και άκυρα) προς τον αριθμό των εδρών (seats) που αναλογούν στην εκλογική περιφέρεια αυτή.
2. Υπολογίζονται πόσοι υποψήφιοι θα εκλεγούν από κάθε κόμμα στην εκλογική περιφέρεια διαιρώντας το σύνολο έγκυρων ψήφων που έλαβε το κόμμα με το εκλογικό μέτρο, με στρογγυλοποίηση προς τα κάτω. Το νούμερο αυτό προστίθεται στο πεδίο electedCount του αντίστοιχου κόμματος στον πίνακα κομμάτων (Parties), ενώ ταυτόχρονα αφαιρείται από τις διαθέσιμες έδρες (πεδίο seats) της εκλογικής περιφέρειας (District).
3. Για κάθε κόμμα (Party) και για κάθε έδρα που αυτό δικαιούται στην εκλογική περιφέρεια εντοπίζονται οι υποψήφιοι (Candidate) με τις περισσότερες ψήφους από το δένδρο υποψηφίων (candidates) του κόμματος. Έστω ότι το πλήθος των υποψηφίων που θα εκλέξει το πρώτο κόμμα είναι <max_party_seats>. Χρησιμοποιούμε έναν **βοηθητικό πίνακα (tempHeap)** με <max_party_seats> στοιχεία, οποίος θα υλοποιήσει έναν **σωρό**. Στον πίνακα αυτό θα αποθηκεύονται δείκτες (Candidates*) προς του κόμβους του δένδρου υποψηφίων (candidates) του κόμματος. Η χρήση του περιγράφεται παρακάτω.

   Για να εντοπισθούν οι υποψήφιοι (candidates) που εκλέγονται σε κάθε εκλογική περιφέρεια (District) θα πρέπει, για κάθε κόμμα (Party), να εφαρμοστεί ο

παρακάτω αλγόριθμος (όπου θεωρούμε πως <pid> είναι το αναγνωριστικό του εκάστοτε κόμματος για το οποίο εφαρμόζεται ο αλγόριθμος):

- Για κάθε περιφέρεια (District), πραγματοποιούνται οι εξής ενέργειες (όπου θεωρούμε ότι <did> είναι το αναγνωριστικό της εκάστοτε περιφέρειας για την οποία εφαρμόζεται ο αλγόριθμος):
    - Έστω ότι <district_party_seats> είναι ο αριθμός των υποψηφίων του κόμματος (Party) με αναγνωριστικό <pid> που εκλέγονται στην περιφέρεια με αναγνωριστικό <did>. Ο βοηθητικός πίνακας (tempHeap) αρχικοποιείται με τους <district_party_seats> πρώτους υποψηφίους του κόμματος που πολιτεύονται σε αυτή την περιφέρεια (σύμφωνα με την ενδοδιατεταγμένη διάσχιση του δένδρου υποψήφιων του κόμματος). Ο πίνακας αυτός υλοποιεί **ουρά προτεραιότητας**, στην οποία η προτεραιότητα καθορίζεται από τον αριθμό των ψήφων που έχουν λάβει οι υποψήφιοι (votes του struct Candidates) που αποθηκεύονται στον πίνακα (ζητείται δηλαδή να υλοποιηθεί **σωρός**).
    - Πραγματοποιείται διάσχιση του δένδρου υποψηφίων του κόμματος (Party) (candidates) με αναγνωριστικό <pid> και για κάθε υποψήφιο που πολιτεύεται στην περιφέρεια με αναγνωριστικό <did>, εξετάζεται αν ο αριθμός των ψήφων του είναι μεγαλύτερος από αυτών του υποψηφίου με τις λιγότερες ψήφους στον βοηθητικό πίνακα (tempHeap). Αν συμβαίνει αυτό τότε αντικαθιστούμε τον υποψήφιο με τις λιγότερες ψήφους στον tempHeap με τον τρέχοντα υποψήφιο από το δένδρο υποψηφίων (candidates) του κόμματος. Η αντικατάσταση γίνεται έτσι ώστε ο βοηθητικός πίνακας (tempHeap) να εξακολουθεί να είναι σωρός.
    - Μετά το τέλος της διάσχισης του δένδρου υποψηφίων (candidates) του κόμματος, ο βοηθητικός πίνακας (tempHeap) θα περιέχει δείκτες στους κόμβους του δένδρου υποψηφίων (candidates) του κόμματος που αντιστοιχούν σε όσους υποψήφιους εκλέγονται στην εκλογική περιφέρεια (District) με αναγνωριστικό <did>. Για κάθε έναν από αυτούς τους υποψήφιους , θέτουμε το πεδίο isElected της αντίστοιχης εγγραφής στην τιμή true.

Μετά το πέρας της εκτέλεσης ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
M <did>
seats
<cid₁> <pid₁> <votes₁>,
<cid₂> <pid₂> <votes₂>,
…,
<cidₙ> <pidₙ> <votesₙ>
DONE
```

όπου n είναι ο αριθμός των εδρών στην εκλογική περιφέρεια με αναγνωριστικό <did> και για κάθε i ∈ {1, …, n}, <cidi>, <pidi>, <votesi> είναι το αναγνωριστικό, το κόμμα, και ο αριθμός ψήφων του i-οστού υποψηφίου που εκλέγεται στην εκλογική περιφέρεια, αντίστοιχα.

– **G**

Γεγονός τύπου *form government* το οποίο σηματοδοτεί την απόδοση των λοιπών αδιάθετων εδρών των εκλογικών περιφερειών.

Βρίσκουμε το **πρώτο κόμμα**, το οποίο χάριν απλοποίησης της εργασίας, είναι το κόμμα που έχει συγκεντρώσει τις περισσότερες έδρες σε ολόκληρη την επικράτεια (δηλαδή είναι το κόμμα με τη μεγαλύτερη τιμή στο πεδίο electedCount του πίνακα κομμάτων (Parties)).

Διανύουμε τον πίνακα εκλογικών περιφερειών (Districts), και σε κάθε περιφέρεια, όσες έδρες περισσεύουν προσμετρούνται στο πρώτο κόμμα (ο αριθμός των εδρών αυτών δίνεται από την τιμή του πεδίου seats). Έστω ότι, τον αριθμό των εδρών που περισσεύουν για το κόμμα με αναγνωριστικό <pid>, τον ονομάζουμε *remainingSeats*.

Για το κόμμα με αναγνωριστικό <pid>, θα πρέπει να εφαρμοστεί και πάλι η διαδικασία που περιγράφεται στο βήμα 3 του γεγονότος M. Ωστόσο, αυτή τη φορά θα πρέπει να αγνοηθούν όσοι υποψήφιοι έχουν ήδη οριστεί ως εκλεγμένοι βουλευτές (δηλαδή έχουν την τιμή true στο πεδίο isElected), έτσι ώστε να εκλεγούν μόνο οι επιπρόσθετοι υποψήφιοι που απαιτούνται ανά περιφέρεια. Επιπλέον προσθέτουμε την τιμή του *remainingSeats* στο πεδίο electedCount του κόμματος με αναγνωριστικό <pid>.

Μετά το πέρας της εκτέλεσης ενός τέτοιου γεγονότος, το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
G
seats
<cid1> <pid1> <votes1>,
<cid2> <pid2> <votes2>,
…,
<cidn> <pidn> <votesn>
DONE
```

όπου n είναι ο συνολικός αριθμός αδιάθετων εδρών σε όλες τις εκλογικές περιφέρειες (Districts) και για κάθε $i \in \{1, \ldots, n\}$, $<did_i>$, $<cid_i>$, $<votes_i>$ είναι το αναγνωριστικό της εκλογικής περιφέρειας, το αναγνωριστικό του i-οστού υποψηφίου που εκλέγεται στη συγκεκριμένη εκλογική περιφέρεια και το πλήθος των ψήφων που έλαβε, αντίστοιχα.

– **N**

Γεγονός τύπου *form parliament* το οποίο σηματοδοτεί την εκλογή των μελών της Βουλής.

Αρχικά, θα πρέπει **για κάθε κόμμα, να γίνει διάσχιση του δένδρου υποψήφιων του κόμματος και να διαγραφούν όλα τα στοιχεία του δένδρου που αντιστοιχούν σε υποψηφίους του κόμματος που δεν εκλέγονται**. Η διαγραφή όλων των υποψήφιων αυτών θα πρέπει να γίνει με μια μόνο διάσχιση κάθε τέτοιου δένδρου. Τα δένδρα που προκύπτουν μετά τις διαγραφές περιέχουν μόνο εκείνους τους υποψηφίους του κόμματος που έχουν εκλεγεί.

Στη συνέχεια, θα πρέπει να συνενώνονται τα δέντρα αυτά, (ακολουθώντας τον αλγόριθμο που ζητείται στην **Άσκηση 2 β) της 3ης Σειράς Ασκήσεων**) ώστε να δημιουργηθεί η λίστα βουλευτών, δηλαδή η δομή κοινοβουλίου (Parliament) . Το κοινοβούλιο θα πρέπει να είναι ταξινομημένο σε φθίνουσα σειρά ως προς το αναγνωριστικό <cid> του κάθε βουλευτή. Η

χρονική πολυπλοκότητα της συνένωσης θα πρέπει να είναι O(n), όπου n είναι ο συνολικός αριθμός των βουλευτών που εκλέγονται.

Στο τέλος του γεγονότος αυτού, τα δέντρα υποψηφίων του κάθε κόμματος θα πρέπει να περιέχουν μόνο τους εκλεγμένους βουλευτές. Μετά το πέρας της εκτέλεσης ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
N
members
<cid1> <pid1> <votes1>,
<cid2> <pid2> <votes2>,
…,
<cidn> <pidn> <votesn>
DONE
```

όπου n είναι ο αριθμός των εκλεγμένων βουλευτών και για κάθε $i \in \{1, \ldots, n\}$, $<cid_i>$, $<pid_i>$, $<votes_i>$ είναι το αναγνωριστικό του i-οστού βουλευτή, το κόμμα στο οποίο ανήκει, και ο αριθμός των ψήφων που αυτός έλαβε, αντίστοιχα.

– **I <did>**

Γεγονός τύπου *print district* το οποίο σηματοδοτεί την εκτύπωση στοιχείων από την εκλογική περιφέρεια με αναγνωριστικό <did>. Το γεγονός απαιτεί την αναζήτηση της εκλογικής περιφέρειας στον πίνακα εκλογικών περιφερειών. Κατά την εκτέλεση ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
I <did>
seats
<seats>
blanks
<blanks>
invalids
<invalids>
partyVotes
<pid1> <votes1>,
<pid2> <votes2>,
…,
<pidk> <votesk>
DONE
```

όπου <seats> είναι ο αριθμός των εδρών που εκλέγει αυτή η εκλογική περιφέρεια, <blanks> είναι ο αριθμός των λευκών και <invalids> είναι ο αριθμός των άκυρων ψήφων

στην εκλογική περιφέρεια. Επίσης k είναι ο αριθμός των κομμάτων που συμμετέχουν στις εκλογές και για κάθε i ∈ {1, …, k}, <pid_i> είναι το αναγνωριστικό του i -οστού κόμματος και <$votes_i$> ο αριθμός των ψήφων που έλαβε το κόμμα με αναγνωριστικό <pid_i> στην περιφέρεια με αναγνωριστικό <did>.

– **J <sid>**

Γεγονός τύπου *print station* το οποίο σηματοδοτεί την εκτύπωση στοιχείων από το εκλογικό τμήμα με αναγνωριστικό <sid>. Κατά την εκτέλεση ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
J <sid>
registered
n
voters
<vid1> <voted1>,
<vid2> <voted2>,
…,
<vidn> <votedn>
DONE
```

όπου n είναι ο αριθμός των εγγεγραμμένων ψηφοφόρων στο εκλογικό τμήμα με αναγνωριστικό <sid> και για κάθε i ∈ {1, …, n}, <vid_i> και <$voted_i$> είναι το αναγνωριστικό του i-οστού ψηφοφόρου και το αν ο ψηφοφόρος έχει ασκήσει το εκλογικό του δικαίωμα, αντίστοιχα.

– **K <pid>**

Γεγονός τύπου *print party* το οποίο σηματοδοτεί την εκτύπωση στοιχείων από το κόμμα με αναγνωριστικό <pid>. Κατά την εκτέλεση ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
K <pid>
elected
<cid1> <votes1>,
<cid2> <votes2>,
…,
<cidn> <votesn>
DONE
```

όπου n είναι ο αριθμός των εκλεγμένων υποψηφίων του κόμματος και για κάθε i ∈ {1, …, n}, <cid_i> είναι το αναγνωριστικό του i-οστού υποψηφίου στο δένδρο των υποψήφιων του κόμματος, στη διάταξη που προκύπτει αν εφαρμοστεί ενδοδιατεταγμένη διάσχιση στο δένδρο και <$votes_i$> είναι ο αριθμός των ψήφων που έλαβε ο υποψήφιος αυτός.

– **L**

Γεγονός τύπου *print parliament* το οποίο σηματοδοτεί την εκτύπωση των εκλεγμένων βουλευτών επικράτειας. Κατά την εκτέλεση ενός τέτοιου γεγονότος το πρόγραμμα θα πρέπει να τυπώνει την ακόλουθη πληροφορία:

```
L
members
<cid1> <pid1> <votes1>,
<cid2> <pid2> <votes2>,
…,
<cidn> <pidn> <votesn>
DONE
```

όπου n είναι ο αριθμός των εκλεγμένων βουλευτών και για κάθε $i \in \{1, \ldots, n\}$, $<cid_i>$, $<pid_i>$, $<votes_i>$ το αναγνωριστικό, το κόμμα, και ο αριθμός των ψήφων που έλαβε ο i-οστός βουλευτής αντίστοιχα.

Αρχικές τιμές πεδίων δομών δεδομένων

- Πίνακας εκλογικών περιφέρειών **(**Districts):
  - **data**: κάθε κελί αρχικοποιείται ως District (περιγραφή παρακάτω)
- Εγγραφή εκλογικής περιφέρειας **(**District):
  - **did**: λήψη από testfile. Για κενό κελί: -1.
  - **seats**: λήψη από testfile.
  - **blanks**: 0.
  - **invalids**: 0.
  - **partyVotes**: 0 σε κάθε κελί.
- Πίνακας κατακερματισμού εκλογικών τμημάτων (Stations):
  - **capacity:** επιλογή από εσάς.
  - **data:** κατασκευάζεται δυναμικά από εσάς. κάθε κελί αρχικοποιείται ως StationsList (περιγραφή παρακάτω)
- Ταξινομημένη αλυσίδα εκλογικών τμημάτων (StationsList):
  - **begin:** NULL.
- Εγγραφή εκλογικού τμήματος (Station):
  - **sid:** λήψη από testfile.
  - **did:** λήψη από testfile.
  - **registered:** 0.
  - **voters.root:** NULL.
  - **next:** NULL.
- Εγγραφή ψηφοφόρου (Voter):
  - **sid:** λήψη από testfile.
  - **voted:** false.
  - **registered:** 0.
  - **parent:** NULL.
  - **lc:** NULL
  - **rc:** NULL.
- Πίνακας κομμάτων **(**Parties):
  - **data**: κάθε κελί αρχικοποιείται ως Party (περιγραφή παρακάτω)
- Εγγραφή κόμματος (Party):
  - **pid:** αριθμός αντίστοιχου κελιού του πίνακα κομμάτων (Parties)
  - **electedCount:** 0
  - **candidates.root:** NULL
- Εγγραφή υποψήφιου βουλευτή (Candidate):
  - **cid:** λήψη από testfile.
  - **pid:** -1.

    - **did:** λήψη από testfile.
    - **votes:** 0.
    - **isElected:** false.
    - **lc:** NULL
    - **rc:** NULL
- Κοινοβούλιο (Parliament):
    - **begin:** NULL.
- Εγγραφή εκλεγμένου βουλευτή (ElectedCandidate):
    - **cid:** αντιγραφή πεδίου από αντίστοιχο candidate.
    - **pid:** αντιγραφή πεδίου από αντίστοιχο candidate.
    - **did:** αντιγραφή πεδίου από αντίστοιχο candidate.
    - **next:** NULL.

Βαθμολογία

A	announce elections	1
D	create district	2
S	create station	10
C	register candidate	10
R	register voter	10
V	vote	10
M	count votes	20
G	form government	20
N	form parliament	13
I	print district	1
J	print station	1
K	print party	1
L	print parliament	1

Πληροφορίες Παράδοσης

- Σε ένα ζευγάρι αρχείων πηγαίου κώδικα (πχ voting.c, voting.h) θα σας δοθούν οι ορισμοί των κατάλληλων τύπων για την υλοποίηση των ζητούμενων δομών, οι ορισμοί των κατάλληλων καθολικών μεταβλητών, οι υπογραφές των συναρτήσεων γεγονότων. Θα υλοποιήσετε την εργασία σας επεκτείνοντας τα αρχεία αυτά με κατάλληλο τρόπο.
- Κώδικας για parsing των γεγονότων από ένα input file θα σας δοθεί στο main source file (main.c). Η κλήση των γεγονότων πραγματοποιείται, επίσης, στο σώμα της main.
- Θα σας δοθεί ένα Makefile που χτίζει το εκτελέσιμο αρχείο από τα παραπάνω αρχεία πηγαίου κώδικα.
- Το παραδοτέο σας θα πρέπει να περιέχει μόνο τα παραπάνω αρχεία πηγαίου κώδικα και το Makefile. Δεν μπορείτε να προσθέσετε επιπλέον αρχεία.

Μπορείτε να χτίσετε και στη συνέχεια να τρέξετε το εκτελέσιμο της εργασίας σας με τις ακόλουθες εντολές

```
make
./main
```

Τέλος, δίνονται τα γεγονότα για την αρχικοποίηση παραμέτρων υλοποίησης του **πίνακα κατακερματισμού εκλογικών τμημάτων (Stations)**

- **BB MaxStationsCount <ParsedMaxStationsCount>**
- **BB MaxSid <ParsedMaxSid>**